

Fundamentals of Subprograms

- **A subprogram is a piece of code that can be used over and over again. If you were coding a calculator, you could make a subprogram that only does adding. Every time you need to do adding, you can use the same subprogram**

General Subprogram Characteristics

- Each subprogram has a single entry point.
- The calling program unit is suspended during the execution of the called subprogram, which implies that there is only one subprogram in execution at any given time.
- Control always returns to the caller when the subprogram execution terminates.

Here is a simple subprogram:

```
def newYear():  
    print("Happy new year!")  
newYear()
```

Basic Definitions

- **subprogram call** :is the explicit request that a specific subprogram be executed. A
- **subprogram is said to be active** :if, after having been called, it has begun execution but has not yet completed that execution.

- **A subprogram header**, which is the first part of the definition, serves several purposes.

1. First, it specifies that the following syntactic unit is a subpro- gram definition of some particular kind.
2. In languages that have more than one kind of subprogram, the kind of the subprogram is usually specified with a special word.
3. it may optionally specify a list of parameters.

- Consider the following header examples:

def adder (parameters):

This is the header of a Python subprogram named adder

- **Ruby subprogram headers** also begin with def. The header of a JavaScript subprogram begins with function.
- **In C**, the header of a function named adder might be as follows:
- **void adder (parameters) or data type adder(parameters)**
- The reserved word void in this header indicates that the subprogram does not return a value.
- **The body of subprograms defines its actions.**
- In the C-based languages (and some others—for example, JavaScript) the body of a subprogram is delimited by braces.
- **In Ruby**, an end statement terminates the body of a subprogram.
- As with compound statements, the statements in the body of a Python function must be indented and the end of the body is indicated by the first statement that is not indented.

python subprogram

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Emil")
```


- The **parameter profile of a subprogram** contains the number, order, and types of its formal parameters.
- The **protocol of a subprogram** is its parameter profile plus, if it is a function, its return type. In languages in which subprograms have types, those types are defined by the subprogram's protocol.
- **Subprograms can have declarations as well as definitions.** This form parallels the variable declarations and definitions in C, in which the declarations can be used to provide type information but not to define variables. Subprogram declarations provide the subprogram's protocol but do not include their bodies
- Function declarations are common in C and C++ programs, where they are called prototypes. Such declarations are often placed in header files.
- In most other languages (other than C and C++), subprograms do not need declarations, because there is no requirement that subprograms be defined before they are called.

c language function

```
#include <stdio.h>
```

```
/* function statement */
```

```
int add(int a, int b);
```

```
/* function definition */
```

```
int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

```
int main() {
```

```
    int result = add(2, 3);
```

```
    printf("The result is %d\n", result);
```

```
    return 0;
```

```
}
```

Parameters

- There are two ways that a non- method subprogram can gain access to the data that it is to process:
- **through direct access to nonlocal variables** (declared elsewhere but visible in the sub- program) or **through parameter passing**.
- Data passed through parameters are accessed through names that are local to the subprogram. Parameter passing is more flexible than direct access to nonlocal variables. In essence, a subprogram with parameter access to the data that it is to process is a parameterized computation
- **.parameterized computation:** It can perform its computation on whatever data it receives through its parameters (presuming the types of the parameters are as expected by the subprogram).
- If **data access is through nonlocal variables**, the only way the computation can proceed on different data is to assign new values to those nonlocal variables between calls to the subprogram.

data access is through nonlocal variables,

```
#include <stdio.h>
int a = 20; // Global declaration
void global();
void main() {
printf("In main() function a = %d\n", a); // Prints 20
global();
a = a + 15; // Uses global variable
printf("In main() function a = %d\n", a); // Prints 35
}
void global() {
a = a + 20; // Uses global variable
printf("In global() function a = %d\n", a); // Prints 55
}
```

parameters

- The parameters in the subprogram header are called **formal parameters**. They are sometimes thought of as dummy variables because they are not variables in the usual sense: In most cases, they are bound to storage only when the subprogram is called, and that binding is often through some other program variables.
- Subprogram call statements must include the name of the subprogram and a list of parameters to be bound to the formal parameters of the subprogram. These parameters are called **actual parameters**.

positional parameters.

- In nearly all programming languages, the correspondence between actual and formal parameters—or the binding of actual parameters to formal parameters—is done by position: The first actual parameter is bound to the first formal parameter and so forth. Such parameters are called positional parameters. This is an effective and safe method of relating actual parameters to their corresponding formal parameters, as long as the parameter lists are relatively short.

```
def nameAge(name, age):  
    print("Hi, I am", name)  
    print("My age is ", age)  
print("Case-1:")  
nameAge("Prince", 20)
```

```
output:  
Case-1:  
Hi, I am Prince  
My age is 20
```

keyword parameters,

- Keyword-only arguments mean whenever we pass the arguments(or value) by their parameter names at the time of calling the function in Python in which if you change the position of arguments then there will be no change in the output.

```
def nameAge(name, age):  
    print("Hi, I am", name)  
    print("My age is ", age)  
nameAge(age=20, name="Prince")
```

- In Python, Ruby, C++, Fortran 95+ Ada, and PHP, formal parameters can have default values. A default value is used if no actual parameter is passed to the formal parameter in the subprogram header.
- Consider the following Python function header:

```
def compute_pay(income, exemptions = 1, tax_rate)
```

The exemptions formal parameter can be absent in a call to `compute_pay`; when it is, the value 1 is used. No comma is included for an absent actual parameter in a Python call, because the only value of such a comma would be to indicate the position of the next parameter, which in this case is not necessary because all actual parameters after an absent actual parameter must be keyworded. For example, consider the following call:

```
pay = compute_pay(20000.0, tax_rate = 0.15)
```


- In C++, which does not support keyword parameters, the rules for default parameters are necessarily different. The default parameters must appear last, because parameters are positionally associated. Once a default parameter is omitted in a call, all remaining formal parameters must have default values. A C++ function header for the `compute_pay` function can be written as follows:

**`float compute_pay(float income, float tax_rate,
int exemptions = 1)`**

- Notice that the parameters are rearranged so that the one with the default value is last. An example call to the C++ `compute_pay` function is
- `pay = compute_pay(20000.0, 0.15);`
- In most languages that do not have default values for formal parameters, the number of actual parameters in a call must match the number of formal parameters in the subprogram definition header.

Design Issues for Subprograms

- One obvious issue is the choice of one or more parameter-passing methods that will be used.
- A closely related issue is whether the types of actual parameters will be type checked against the types of the corresponding formal parameters.
- The most important question here is whether local variables are statically or dynamically allocated.
- Next, there is the question of whether subprogram definitions can be nested. Another issue is whether subprogram names can be passed as parameters. If subprogram names can be passed as parameters and the language allows subprograms to be nested, there is the question of the correct referencing environment of a subprogram that has been passed as a parameter.
- Finally, there are the questions of whether subprograms can be overloaded or generic. An overloaded subprogram is one that has the same name as another subprogram in the same referencing environment. A generic subprogram is one whose computation can be done on data of different types in different contexts. Another issue is whether subprograms can be nested within functions.

- **Overloaded and generic subprograms are both types of subprograms, but they differ in how they handle parameters:**
- **Overloaded subprograms :**Have the same name as another subprogram, but differ in their parameter list and/or return type.
- **Generic subprograms:**Take parameters of different types on different activations.
- Overloaded subprogram handles the function call according to the passed parameters.
- Whereas generic subprograms are able to accept as many parameter and then able to handle the passed data according to it.

Overloaded subprograms

To calculate area, let's say we created a function/subprogram/subroutine, named as AREA, and the passed parameters will be like length breadth side radius

So overloaded subprogram prototype will look like

AREA(int side);

AREA(int length,int Breadth);

AREA(int radius);

.

The above function name is same, but during the runtime, the program will decide which specific subroutine to call according to parameters passed.

generic subprogram

Now when we declare a generic subprogram , it is kind of like universal solution for a problem.

With the same example of area

AREA (param, param, param,type);

The Name of param is x,y,z with type int. And the last param type is passed along with the call to indicate what we want to do with it or what kind of thing was passed , let's say this type declared as char , so passing 'c' will be like circle , 's' like square , 'r' for rectangle and so on. In this case you will have to write this subroutine in such a way that it will figure out what to do when it is called.

Design Issues for Subprograms

- • Are local variables statically or dynamically allocated?
- • Can subprogram definitions appear in other subprogram definitions?
- • What parameter-passing method or methods are used?
- • Are the types of the actual parameters checked against the types of the formal parameters?
- • If subprograms can be passed as parameters and subprograms can be nested, what is the referencing environment of a passed subprogram?
- • Can subprograms be overloaded?
- • Can subprograms be generic?

Local Referencing Environments

Local Variables

Subprograms can define their own variables, thereby defining local referencing environments. Variables that are defined inside subprograms are called local variables, because their scope is usually the body of the subprogram in which they are defined

- local variables can be either static or stack dynamic
- **Static Instances**
- **Definition:** Memory for static instances is allocated at compile time and persists for the lifetime of the program.
- **Scope:** They can be global, local (within a function but declared static), or class-level (static members).
- **Lifetime:** They exist until the program terminates.
- Example: `static int staticVar = 10; // Static variable`

- **Stack Dynamic Instances**

- Definition: Memory for stack dynamic instances is allocated on the stack at runtime, typically when a function is called, and deallocated when the function exits.
- Scope: Limited to the block in which they are defined (e.g., a function or a loop).
- Lifetime: Exists only while the function is executing, and is automatically cleaned up when the function returns.
- Example:

```
void myFunction() {  
    int stackVar = 20; // Stack dynamic variable  
}
```


- There are several **advantages of stack-dynamic local variables**, the primary one being the flexibility they provide to the subprogram. It is essential that recursive subprograms have stack-dynamic local variables. Another advantage of stack-dynamic locals is that the storage for local variables in an active subprogram can be shared with the local variables in all inactive subprograms.
- The main **disadvantages of stack-dynamic local variables** are the following: First, there is the cost of the time required to allocate, initialize (when necessary), and deallocate such variables for each call to the subprogram. Second, accesses to stack-dynamic local variables must be indirect, whereas accesses to static variables can be direct.⁴ This indirectness is required because the place in the stack where a particular local variable will reside can be determined only during execution

- The **primary advantage of static local variables over stack-dynamic local variables** is that they are slightly more efficient—they require no run-time overhead for allocation and deallocation. Also, if accessed directly, these accesses are obviously more efficient
- The **greatest disadvantage of static local variables** is their inability to support recursion. Also, their storage cannot be shared with the local variables of other inactive subprograms.

- In most contemporary languages, local variables in a subprogram are by default stack dynamic. In C and C++ functions, locals are stack dynamic unless specifically declared to be static. For example, in the following C (or C++) function, the variable `sum` is static and `count` is stack dynamic.

```
int adder(int list[], int listlen) {  
    static int sum = 0;  
    int count;  
    for (count = 0; count < listlen; count++) sum += list [count];  
    return sum;  
}
```

- **The methods of C++, Java, and C# have only stack-dynamic local variables.**

- In Python, the only declarations used in method definitions are for globals. Any variable declared to be global in a method must be a variable defined outside the method. A variable defined outside the method can be referenced in the method without declaring it to be global, but such a variable cannot be assigned in the method. If the name of a global variable is assigned in a method, it is implicitly declared to be a local and the assignment does not disturb the global. All local variables in Python methods are stack dynamic.

```
def greet():  
    message = 'Hello'  
    print('Local', message)  
greet()  
print(message)
```

Output
Local Hello
NameError: name
'message' is not defined

python global variable

```
message = 'Hello'  
def greet():  
    print('Local', message)  
greet()  
print('Global', message)
```

Output

Local Hello

Global Hello

Nested Subprograms

- . Among these are JavaScript, Python, Ruby, and Lua. Also, most functional programming languages allow subprograms to be nested.
- Some programming languages, like C, C++, and Java, don't support nested functions because it can make compilers more difficult to implement. However, some compilers do support nested functions as an extension.
- **Nested function is not supported by C because we cannot define a function within another function in C.** We can declare a function inside a function, but it's not a nested function

```
// C program to illustrate the concept of Nested function.
```

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Main");
```

```
    int fun()
```

```
    {
```

```
        printf("fun");
```

```
        // defining view() function inside fun() function.
```

```
        int view()
```

```
        {
```

```
            printf("view");
```

```
        }
```

```
        return 1;
```

```
    }
```

```
    view();
```

```
}
```

Output:

Compile time error: undefined reference to `view'

- A function defined inside another function is known as a nested function. Nested functions can access variables of the enclosing scope.
- Nested functions are used so that they can be protected from everything happening outside the function. This process is also known as Encapsulation.

Python program to illustrate nested functions

```
def outerFunction(text):
```

```
    text = text
```

```
    def innerFunction():
```

```
        print(text)
```

```
    innerFunction()
```


Parameter-Passing Methods

- Parameter-passing methods are the ways in which parameters are transmitted to and/or from called subprograms

Semantics Models of Parameter Passing

- Formal parameters are characterized by one of three distinct semantics models:
- (1) They can receive data from the corresponding actual parameter;
- (2) they can transmit data to the actual parameter; or (3) they can do both.
- These models are called in mode, out mode, and inout mode, respectively.
- For example, consider a subprogram that takes two arrays of int values as parameters—list1 and list2. The subprogram must add list1 to list2 and return the result as a revised version of list2. Furthermore, the subprogram must create a new array from the two given arrays and return it. For this subprogram, list1 should be in mode, because it is not to be changed by the subprogram. list2 must be inout mode, because the subprogram needs the given value of the array and must return its new value. The third array should be out mode, because there is no initial value for this array and its computed value must be returned to the caller.

Modes:

IN: Passes info from caller to the callee.

OUT: Callee writes values in the caller.

IN/OUT: The caller tells the callee the value of the variable, which the callee may update.

Pass By Value

- This method uses in-mode semantics. Changes made to formal parameters do not get transmitted back to the caller.
- Any modifications to the formal parameter variable inside the called function or method affect only the separate storage location and will not be reflected in the actual parameter in the calling environment. This method is also called call by value.

```
#include <stdio.h>
```

```
void swap (int a, int b)
```

```
{
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
int main ()
```

```
{
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    printf ("Before swap, a = %d, b = %d\n", a, b);
```

```
    swap (a, b);
```

```
    printf ("After swap, a = %d, b = %d\n", a, b);
```

```
    return 0;
```

```
}
```

Pass-by-Value

before swap , 10 20
after swap 10 20

Pass-by-Value

- When a parameter is passed by value, the value of the actual parameter is used to initialize the corresponding formal parameter, which then acts as a local variable in the subprogram, thus implementing in-mode semantics.
- Pass-by-value is normally implemented by copy, because accesses often are more efficient with this approach.
- **Advantages:**
- **Safety:** Since a copy of the variable is passed, the original variable remains unchanged. This minimizes side effects, making code easier to understand and debug.
- **Simplicity:** It is often simpler to reason about functions that operate on copies of data. You can be confident that the function will not alter the state of the original argument.
- **Concurrency:** In multi-threaded environments, pass by value can be safer because each thread operates on its own copy of data, reducing the risk of race conditions.

- Use Pass by Value when you want to ensure that the original data remains unchanged and when working with small data types where copying is not costly.

Pass by Reference in C?

- In pass by reference, we pass the address of the variable instead of passing the value of the variable and access the variable using the pointers in the Function. All the changes made to variable in the function will be reflected in the main Program.

Example:

- Consider Pass by reference as google drive and Call by value as bluetooth system. If you receive a file via bluetooth, any changes made to that file will only affect your file and not the main file, which is the core concept of call by reference, whereas any changes made to a file in Google Drive will be reflected everywhere, which is the main ideology behind pass by reference.

```
void swap(int *a,int *b)
{
    int temp=*a;
    *a=*b;
    *b=temp;
}
int main()
{
    int a,b;
    printf("Enter two numbers:");
    scanf("%d %d",&a,&b);
    printf("Numbers Before Swapping: %d and %d\n",a,b);
    swap(&a,&b);
    printf("Numbers After Swapping: %d and %d",a,b);
    return 0;
}
```

before swap 10 20
after swap 20 10

Pass-by-Result

- Pass-by-result is an implementation model for out-mode parameters. When a parameter is passed by result, no value is transmitted to the subprogram. The corresponding formal parameter acts as a local variable, but just before control is transferred back to the caller, its value is transmitted back to the caller's actual parameter, which obviously must be a variable
- Just before control is transferred back to the caller, the value of the formal parameter is transmitted back to the actual parameter.
- . In general, the pass-by-result technique is implemented by copying.

```
void plus(int a, int b, int c)// c is by result
{
    c = a+b;
}
void f()
{
    int x = 3;
    int y = 4;
    int z;
    plus(x, y, z);
}
```

Consider this example where the parameters a and b are pass by value and the parameter c is pass by result. What are the values of x, y and z after the function plus is called? output will be z= 7

Pass by ValueResult”

- Combining both pass by value and pass by result we get the mechanism ”Pass by ValueResult”.
- In this mechanism, the actuals are copied to the formals, the subprogram is executed, and finally the formals are copied back to the actuals.
- Consider the following and determine the value of x after the execution of plus.

```
void plus(int a, int b)
```

```
{
```

```
  b += a;
```

```
}
```

```
void f()
```

```
{
```

```
  int x = 3;
```

```
  plus(4, x);
```

```
}
```

x= 7

Parameter-Passing Methods of Some Common Languages

- C uses pass-by-value. Pass-by-reference (inout mode) semantics is achieved by using pointers as parameters. The value of the pointer is made available to the called function and nothing is copied back. However, because what was passed is an access path to the data of the caller, the called function can change the caller's data. C copied this use of the pass-by-value method from ALGOL 68.
- Ada and Fortran 95 + allow the programmer to specify in mode, out mode, or inout mode on each formal parameter.
- The default parameter-passing method of C# is pass-by-value. Pass-by-reference can be specified by preceding both a formal parameter and its corresponding actual parameter with ref.
- C# also supports out-mode parameters, which are pass-by-reference parameters that do not need initial values.
- The parameter-passing method of Python and Ruby is called pass-by-assignment. Because all data values are objects, every variable is a reference to an object. In pass-by-assignment, the actual parameter value is assigned to the formal parameter. Therefore, pass-by-assignment is in effect pass-by-reference, because the value of all actual parameters are references.

consider the following program fragment:

var a,b : integer;

procedure G(c,d: integer);

begin

c:=c-d;

d:=c+d;

c:=d-c

end;

a:=2;

b:=3;

G(a,b);

If both parameters to G are passed by reference, what are the values of a and b at the end of the above program fragment ?

ans : swapped

Consider the following C program.

```
# include <stdio.h>
void mystery (int *ptr a, int *ptr b) {
    int *temp;
    temp = ptr b;
    ptr b = ptr a;
    ptr a = temp;
}
int main () {
int a = 2016, b=0, c= 4, d = 42;
    mystery (&a, &b);
    if (a < c)
        mystery (&c, &a);
    mystery (&a, &d);
    printf("%d\n", a);
}
```

The output of the program is _____

The mystery about mystery function is it does not affect values in main. As in C, parameters are passed by value- even if they are pointer. So, here the pointer values are exchanged within the function only. (we can use * operator to exchange the values at the location of the pointers and this will affect the values in main).

So, NO CHANGES in a,b,c,d.

And ANSWER is